Autonomous Drone Racing Report

1st Matuszewski Timo
School of Computation, Information and Technology
Technical University of Munich
Munich, Germany

2nd Kenny Jeffrey
School of Engineering and Design
Technical University of Munich
Munich, Germany

***Abstract*—We present an autonomous drone-racing controller based on Model Predictive Contouring Control (MPCC). A quadrotor must fly through a sequence of gates in order while avoiding obstacles, with gate and obstacle poses only revealed once the drone is within sensor range, forcing online replanning. We embed a geometric racing line, produced by a B-spline planner, directly into the prediction model as a function of a progress state, so that the reference cannot "run away" from the drone, a failure mode we observed with a time-indexed reference-tracking Model Predictive Control (MPC). On top of the nominal MPCC we add three lightweight, context-dependent mechanisms: a curvature-based speed cap, a near-gate contouring boost, and a caution mode that slows the drone near unmeasured objects. We also implemented a reinforcement-learning (RL) layer that adapts the MPCC cost weights online, but found training it end-to-end impractical because the embedded solver runs one step at a time; the hand-designed context rules capture the same idea at almost no cost. On a fixed racing track our controller finishes 88% of runs at a $7.37\,\mathrm{s}$ average lap time, and generalises to fully randomised tracks at 87%. Deployed unchanged on the physical Crazyflie, it completed all eight test flights at an $8.70\,\mathrm{s}$ mean lap time, confirming the sim-to-real transfer. An ablation study quantifies the contribution of each component.**

I. Introduction

Autonomous drone racing has become a popular benchmark for agile, high-speed robot control, because it couples fast nonlinear dynamics, tight actuation limits, and hard geometric constraints (the gates) in a single task [1]. Recent work has pushed quadrotors to champion-level performance using both learning-based [2] and optimisation-based [3], [4] approaches, and the gap between the two is an active research direction. Beyond racing, the same problem (flying a prescribed path as fast as possible within dynamics and safety limits) underlies a broad class of agile aerial applications such as inspection and delivery.

In this work we consider the drone-racing task posed by the course simulator: a Crazyflie 2.1 quadrotor must pass four gates in a fixed order and reach the finish, while avoiding four pole obstacles. The challenge is threefold. First, the flight is *time-critical*: the objective is to minimise lap time, which pushes the drone to the limits of what its motors can deliver, where even small tracking errors lead straight to gate-frame collisions. Second, the environment is only *partially observed*: gates and obstacles are known only at nominal poses until the drone comes within a short sensor range, at which point the true pose is revealed and the planned path must be corrected online. Third, the controller must *generalise*: it is ultimately graded on a realistic, hand-set track on real hardware, so overfitting to one geometry is penalised by the sim-to-real gap and by unseen layouts.

We address this with Model Predictive Contouring Control (MPCC) [4]–[7], which optimises progress along a geometric reference path rather than tracking a time-parameterised trajectory. We contribute (i) a complete MPCC realisation for the Crazyflie with soft obstacle and gate-frame avoidance embedded in the cost, (ii) three context-dependent adaptation mechanisms motivated by an RL weight-scheduling idea that we found too expensive to train end-to-end, and (iii) a systematic ablation study on both a fixed and a fully randomised track that isolates the effect of each component.

II. Related Work

Optimisation-based racing. Contouring control was introduced for machining [5] and adapted to autonomous racing of scale cars [6], where progress along a reference is maximised online. Romero et al. transferred MPCC to quadrotors and showed near-time-optimal flight [4], and Krinner et al. extended it with hard safety constraints in MPCC++ [7], while time-optimal planning approaches compute aggressive reference trajectories offline [3], [8]. Our work builds directly on this quadrotor MPCC line [4], [7], but keeps collision avoidance soft in the cost rather than as MPCC++'s hard tunnel, and targets a partially observed, replanning setting rather than a known track.

Learning-based racing and control. End-to-end reinforcement learning has reached champion-level performance from onboard sensing [2], [9], at the cost of large-scale training. A complementary line keeps a model-based controller and learns only part of it: learning the plant model [10] or, as here, the cost-function weights [11]–[13]. We adopt the latter view but, finding end-to-end weight learning too costly with an embedded solver, distil it into interpretable context rules.

III. Problem Formulation

We model the quadrotor as a continuous-time nonlinear system with state $\mathbf{x} \in \mathbb{R}^{12}$ and input $\mathbf{u} \in \mathbb{R}^4$,

$$\mathbf{x} = \left[\mathbf{p}^\top,\, \mathbf{\Phi}^\top,\, \mathbf{v}^\top,\, \boldsymbol{\omega}^\top\right]^\top, \qquad \mathbf{u} = \left[\mathbf{\Phi}_{\mathrm{cmd}}^\top,\, T\right]^\top, \tag{1}$$

where $\mathbf{p}$ is the world position, $\mathbf{\Phi} = (\phi, \theta_{\mathrm{p}}, \psi)$ the roll–pitch–yaw attitude, $\mathbf{v}$ the linear velocity, $\boldsymbol{\omega}$ the body rates, $\mathbf{\Phi}_{\mathrm{cmd}}$ the commanded attitude and T the collective thrust; the continuous dynamics $\dot{\mathbf{x}} = f_{\mathrm{q}}(\mathbf{x}, \mathbf{u})$ follow the reduced attitude-command quadrotor model of [14]. The controller runs at $50\,\mathrm{Hz}$ and

outputs the attitude command $\mathbf{u}$; the simulator integrates the full rigid-body dynamics at 500 Hz.

The track is an ordered set of N_g gates with centre poses $\{\mathbf{g}_i\}_{i=1}^{N_g}$ and N_o cylindrical obstacles. A gate i counts as *passed* when the drone crosses its plane, from the front, inside a $0.45 \times 0.45\,\mathrm{m}$ window. Let $g \in \{0, \dots, N_g - 1, -1\}$ be the index of the next gate to pass, initialised at 0 and set to -1 once the last gate is cleared. A run is a *success* iff $g = -1$ before the horizon of 1500 steps (30 s) elapses. The drone is *disabled* (terminating the episode) on any contact with a gate or obstacle, or on leaving the safety box $\mathcal{X}_{\text{safe}}$.

Crucially, the observation exposes each gate/obstacle at a fixed *nominal* pose until the drone is within a sensor range $r_s = 0.7\,\mathrm{m}$, after which the *true* (randomly perturbed) pose is revealed. The control problem is therefore

$$\begin{aligned} \min_{\mathbf{u}(\cdot)} \;& t_{\text{lap}} \\ \text{s.t.}\;& \dot{\mathbf{x}} = f_{\text{q}}(\mathbf{x}, \mathbf{u}), \quad \mathbf{u} \in \mathcal{U},\ \mathbf{x} \in \mathcal{X}_{\text{safe}}, \\ & \text{pass gates } 1, \dots, N_g \text{ in order}, \\ & \|\mathbf{p} - \mathbf{o}_j\| \geq r_j \ \ \forall j \text{ (collision-free)}, \end{aligned} \tag{2}$$

solved online under partial observability of $\{\mathbf{g}_i\}$ and $\{\mathbf{o}_j\}$.

IV. Methodology

A. System overview

Our controller (Fig. 1) has three stages: a *planner* that builds a smooth, collision-aware geometric racing line through the gates; an *MPCC* that optimises progress along that line while respecting dynamics, input, and soft avoidance constraints; and a set of *context-adaptation* rules that modulate the MPCC speed target and weights from the local track geometry. When any measured object pose changes, the planner replans in a background thread and the new path is swapped into the MPCC.

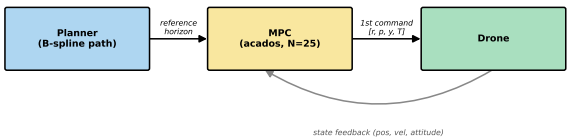

Fig. 1. Control architecture: the planner builds a geometric racing line, the MPCC optimises progress along it, and context rules modulate the speed target and weights. A pose change triggers a background replan.

B. Path planning

For each inter-gate segment we place three fixed waypoints per gate (before, centre, after) along the gate normal, plus $N_f = 4$ free intermediate waypoints whose positions are optimised with L-BFGS-B against a cost that penalises proximity to obstacles and gate frames and deviation from the straight connection. A weighted cubic B-spline is fitted through the waypoints [15] and reparameterised by arc length θ, giving a C^2 reference curve $\mathbf{p}_d(\theta)$ with unit tangent $\mathbf{t}(\theta) = \mathbf{p}_d'(\theta)$. Arc-length parameterisation makes the progress state physically meaningful and decouples the path geometry from the speed profile. Fig. 2 shows a resulting line, with the free waypoints bowed clear of the obstacles. The free waypoints $\{\mathbf{w}_k\}_{k=1}^{N_f}$ of each segment are placed by minimising

$$J_{\text{plan}} = \sum_k \Big(w_o \sum_j \beta(\mathbf{w}_k, \mathcal{C}_j) + w_f\, \Gamma(\mathbf{w}_k) + w_d\, \|\mathbf{w}_k - \bar{\mathbf{w}}_k\|^2 \Big), \tag{3}$$

where β is the obstacle barrier of (7), Γ penalises penetration of any non-target gate frame, and the last term anchors the waypoint to the straight-line position $\bar{\mathbf{w}}_k$ so the line stays short. The problem is solved with L-BFGS-B, warm-started from the previous solution so a replan converges in a few iterations.

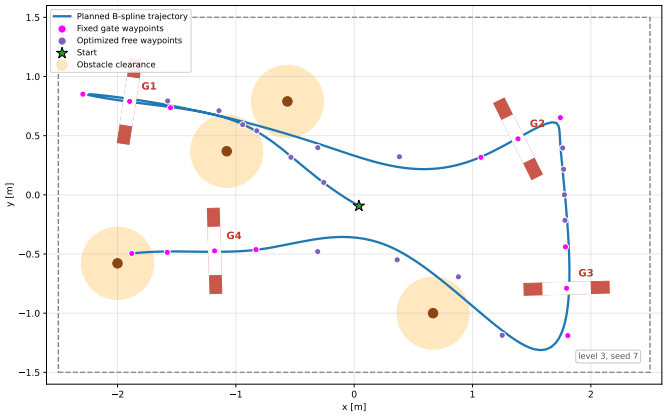

Fig. 2. Top-down view of a planned racing line through the gates, with the free intermediate waypoints bowed away from the obstacles.

C. MPCC formulation

Following [4]–[7] we augment the quadrotor state with a *progress* state θ and its rate v_θ, driven by a virtual input a_θ:

$$\tilde{\mathbf{x}} = [\mathbf{x}^\top, \theta, v_\theta]^\top \in \mathbb{R}^{14}, \quad \tilde{\mathbf{u}} = [\mathbf{u}^\top, a_\theta]^\top \in \mathbb{R}^5, \tag{4}$$

with $\dot{\theta} = v_\theta$, $\dot{v}_\theta = a_\theta$. At a predicted position $\mathbf{p}$ and progress θ we define the path deviation $\mathbf{d} = \mathbf{p} - \mathbf{p}_d(\theta)$ and split it into a *lag* (longitudinal) and a *contouring* (perpendicular) error (Fig. 3)

$$e_l = \mathbf{t}(\theta)^\top \mathbf{d}, \qquad \mathbf{e}_c = \mathbf{d} - e_l\, \mathbf{t}(\theta), \tag{5}$$

and a progress-speed error $e_v = v_\theta - v_{\text{tgt}}$ that rewards fast travel along the path. The MPCC minimises, over the horizon of $N = 25$ nodes with step $\mathrm{d}t = 0.02\,\mathrm{s}$ ($T_p = 0.5\,\mathrm{s}$),

$$\begin{aligned} J = \sum_{k=0}^{N} \Big(& \|\mathbf{e}_{c,k}\|_{q_c}^2 + q_l e_{l,k}^2 + q_v e_{v,k}^2 \\ & + \|\mathbf{\Phi}_k\|_{q_\Phi}^2 + \|\boldsymbol{\omega}_k\|_{q_\omega}^2 + \|\tilde{\mathbf{u}}_k\|_R^2 \\ & + \textstyle\sum_j w_b\, \beta(\mathbf{p}_k, \mathcal{C}_j) + w_g\, \gamma(\mathbf{p}_k) \Big), \end{aligned} \tag{6}$$

expressed as a nonlinear least-squares residual with zero reference. The first line is the core MPCC objective: track

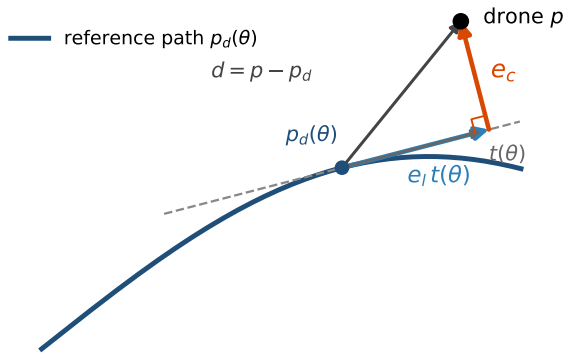

Fig. 3. Contouring formulation: the deviation $d = p - p_d(\theta)$ is split into the lag error e_l along the path tangent $t(\theta)$ and the perpendicular contouring error e_c.

TABLE I
MAIN MPCC WEIGHTS AND SOLVER SETTINGS (FROM `MPCC/CONFIG.PY`).

Symbol	Value	Symbol	Value
q_c (contour)	50	horizon N	25
q_l (lag)	150	step $\mathrm{d}t$	0.02 s
q_v (progress)	5	T_p	0.5 s
q_Φ (att.)	1	w_b (barrier)	5000
q_ω (rate)	5	w_g (ground)	400
$r_\Phi / r_T / r_a$ (effort)	1/50/0.5	$a_{\mathrm{lat}}^{\max}$	20 m/s^2
β_g / ρ (boost)	4 / 0.5 m	$v_{\min}$	0.8 m/s
κ_c / R_c (caution)	0.5 / 1.3 m	V_{tgt}	2.5 m/s

the line ($\mathbf{e}_c$), keep the progress state glued to the drone (e_l), and cruise (e_v). We deliberately set $q_l \gg q_v$ (here 150 vs. 5) so the progress state cannot sprint ahead of the physical drone. The remaining terms penalise attitude, body rate, and control effort. Collision avoidance is *soft*, in contrast to the hard tunnel constraint of MPCC++ [7]: each obstacle and gate frame is modelled as a capsule $\mathcal{C}_j$ and enters the cost through a one-sided barrier residual

$$\beta(\mathbf{p}, \mathcal{C}_j) = \max\!\left(0,\ 1 - \frac{\mathrm{dist}(\mathbf{p}, \mathcal{C}_j)^2}{r_j^2}\right)^2, \tag{7}$$

with a large weight w_b; a further one-sided term γ enforces ground clearance. Keeping avoidance in the cost (rather than as hard constraints) means the QP is always feasible and the gate opening stays free to fly through. State and input box limits ($\mathbf{v}$, v_θ, attitude, thrust, a_θ) are imposed as constraints, with the velocity bounds softened by slack so a transient overspeed never renders the problem infeasible. The full set of weights and solver settings is summarised in Table I.

D. Real-time solver

Problem (6) is transcribed with direct multiple shooting (explicit Runge–Kutta integration) and solved with `acados` [16] using a Gauss–Newton Hessian and the Real-Time Iteration (RTI) scheme [17], which replaces a fully converged SQP solve with a single SQP iteration per control tick; the resulting QP is solved by `HPIPM` [18] in full-condensing mode, with the symbolic model and cost built in CasADi [19]. A receding-horizon warm start shifts the previous solution one node forward, keeping each tick near the previous optimum, so a single iteration suffices and the optimiser converges *across* control ticks rather than within one. On a path swap, where this no longer holds, the solver is re-seeded and re-converged with a short iteration burst. The fixed per-tick cost is what keeps the 50 Hz deadline reliable.

E. Why contouring control instead of reference tracking

Our first controller was a reference-tracking MPC that followed a *time-indexed* trajectory $\mathbf{p}_{\mathrm{ref}}(t_k)$. Because the reference marches forward on a clock regardless of the drone's actual progress, whenever the drone fell behind (e.g. after a replan or an aggressive corner) the MPC chased a far-ahead point at full thrust and the reference effectively "ran away", requiring a hand-tuned reference governor to hold it back. MPCC removes this failure mode structurally: the reference is $\mathbf{p}_d(\theta)$, a function of a *state the optimiser itself controls*, so it can never outrun the drone [4]. This is the central reason we adopted MPCC.

Intuitively, θ is a virtual particle running along the racing line whose speed the optimiser sets, with the drone tethered to it by the contouring and lag costs. Because the far horizon nodes are evaluated at $\mathbf{p}_d(\theta)$ inside an upcoming corner (where a small contouring error at high speed would demand infeasible attitude), the optimiser slows θ before the apex and accelerates out, so MPCC already brakes implicitly through corners [4], [7], with the lag weight $q_l \gg q_v$ keeping θ glued to the drone through the turn. Because our progress reward tracks a target speed rather than maximising raw progress, this implicit braking alone still carries too much speed into sharp turns; the explicit curvature cap of the next subsection lowers that target into corners.

F. Context-dependent adaptation

Two properties of the track are known to the planner but not encoded in the static weights: how sharp the upcoming corner is, and how close the drone is to a gate. We exploit both.

Curvature speed cap. From the arc-length spline we compute the path curvature $\kappa(s) = \|\mathbf{p}_d''(s)\|$, smooth it over a short window, and set a lateral-acceleration-limited speed cap

$$v_{\mathrm{cap}}(s) = \mathrm{clip}\!\left(\sqrt{a_{\mathrm{lat}}^{\max}/\kappa(s)},\ v_{\min},\ V_{\mathrm{tgt}}\right), \tag{8}$$

taking the tightest cap over a 1.5 m look-ahead so the drone brakes *before* the apex. The MPCC speed target v_{tgt} in e_v is set to this cap (Fig. 4).

Near-gate contouring boost. The drone tends to drift laterally exactly at a gate and clip the frame. For any prediction node whose progress lies within a window ρ of a gate centre θ_{g_i} we multiply the contouring weight,

$$q_c \leftarrow \beta_g\, q_c \quad \text{if} \quad \min_i |\theta - \theta_{g_i}| \le \rho, \tag{9}$$

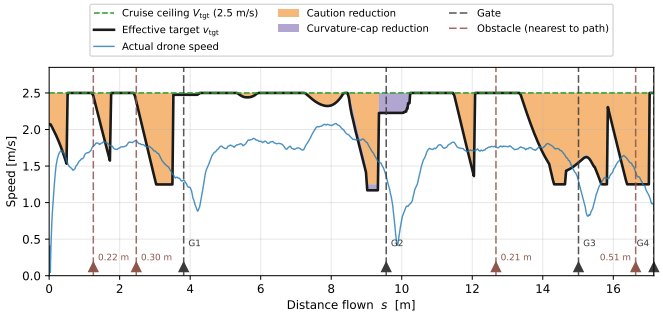

Fig. 4. Speed along a level-3 run: starting from the cruise ceiling V_{tgt}, the effective target v_{tgt} is lowered by the curvature cap $v_{\mathrm{cap}}(s) = \sqrt{a_{\mathrm{lat}}^{\max}/\kappa(s)}$ (curvature-cap reduction) and by the caution scaling near not-yet-revealed gates and obstacles (caution reduction), while straights keep the full cruise speed. The actual drone speed tracks this modulated target; gates (G1–G4) and the nearest obstacle to the path are marked.

($\beta_g = 4$, $\rho = 0.5\,\mathrm{m}$), pulling the prediction tightly onto the gate-centred line where precision matters, while straights keep the loose, fast baseline weighting.

Caution mode. While the true pose of a gate/obstacle is still unknown, the target speed is scaled down toward a fraction κ_c of V_{tgt} as the drone closes within a radius R_c of it, so that it is already slow when the true pose is revealed at r_s and the path is corrected (caution reduction in Fig. 4).

G. Online replanning

A key departure from the prior MPCC racing work [4], [7], which plans a single reference around a fully known track and then only tracks it, is that we replan the reference online during the run as the track is revealed. A change in any measured gate/obstacle pose (detected when the true pose is revealed within r_s) triggers a full-track replan in a background thread, so the $50\,\mathrm{Hz}$ loop never stalls. When the new path is ready it is swapped in, the embedded spline coefficients are refreshed, and the progress state is re-localised by projecting the drone onto the new arc length. A gate-progress barrier additionally prevents θ from advancing past the current target gate until the pass is confirmed, so the optimiser cannot "skip" a gate. Concretely, the per-node progress anchor is clamped to

$$\theta_k \le \theta_g + \Delta_{\mathrm{lead}}, \qquad \Delta_{\mathrm{lead}} = 0.4\,\mathrm{m}, \tag{10}$$

where θ_g is the arc length of the current target gate; the bound only advances to the next gate once the environment confirms the pass.

H. Learned weight adaptation (attempted)

The three rules above are hand-designed instances of a more general idea: schedule the MPCC cost weights online from context. We therefore implemented a Weights-varying MPC in the spirit of [11], where a policy outputs, every few ticks, bounded multipliers on grouped baseline weights

$$m = \exp\big(\ln(4)\cdot\tanh(a)\big) \in [\tfrac{1}{4}, 4], \quad a \in \mathbb{R}^6, \tag{11}$$

(so $a = 0$ recovers the exact baseline), from a 27-dimensional feature vector (tracking errors, speed, look-ahead curvature, and relative gate/obstacle geometry). We trained it with PPO [20] with the MPCC *inside* the environment loop, and a reward combining path progress, gate bonuses, and crash/effort penalties; this places our work in the family of RL-tuned MPC [12], [13]. In practice, training was impractical on our hardware: because `acados` is a sequential C solver, the environment cannot be vectorised like a pure simulator, so each rollout runs one MPCC at a time and data collection is slow. Rather than pay that training cost, we distilled the same context-dependent intuition into the deterministic curvature-, gate-, and caution rules of (8)–(9), which require no training and are trivially interpretable. The policy interface remains in the code (disabled by default) for future work.

TABLE II
NOMINAL PERFORMANCE ($n = 100$, PER LEVEL).

Track	Success	Mean time [s]	Median [s]
Level 2 (fixed)	88.0%	7.37	7.30
Level 3 (randomised)	87.0%	8.51	8.50

V. RESULTS

A. Experimental setup

We evaluate in the course simulator (`crazyflow`, a JAX/MuJoCo-MJX Crazyflie 2.1 model). Two settings are used: *Level 2*, a single fixed, hand-designed four-gate track with per-episode position/attitude jitter ($\pm 0.15\,\mathrm{m}$ on gates/obstacles, plus inertial and actuation disturbances); and *Level 3*, identical disturbances but with the whole track geometry randomised every episode. All numbers below are over $n = 100$ episodes at a *fixed* seed (seed $= 0$): because the seed is held constant across configurations, every variant is evaluated on the *same* sequence of disturbances (and, on Level 3, the same sequence of random tracks), so the comparisons are *paired*. We treat Level 2 as the *development* track (on which the controller was tuned) and Level 3 as a *held-out* test of generalisation. A run is a success if all four gates are passed. We report success rate, the mean/median lap time over *successful* runs, and a breakdown of failures by the nearest physical object at the crash.

B. Nominal performance

Table II reports the full controller. On the fixed track it finishes 88% of runs at $7.37\,\mathrm{s}$ mean lap time, and, importantly, it retains 87% on the fully randomised track, indicating that the method does not overfit the single Level 2 geometry. The higher Level 3 mean lap time ($8.51\,\mathrm{s}$) chiefly reflects the randomised tracks being on average longer than the fixed Level 2 layout, not a slower or more conservative controller. Failures are dominated by gate-frame clips at the tight 180° reversal (gate 2) and at the final gate, where a late pose reveal leaves the least room to correct.

C. Real-world deployment

The ultimate test is the physical Crazyflie. We deployed the *exact* controller (config `final.toml`, $V_{\mathrm{tgt}} =$

TABLE III
REAL-WORLD DEPLOYMENT ON THE PHYSICAL CRAZYFLIE (`final.toml`, $V_{\text{tgt}} = 2.5\,\text{m/s}$): EIGHT CONSECUTIVE COMPLETED FLIGHTS.

Metric	Value
Completed flights	8/8
Mean lap time	8.70 s
Median lap time	8.71 s
Best / worst	8.20 / 9.24 s
Std. deviation	0.30 s
Level-3 simulation reference	8.51 s

TABLE IV
ABLATION ON LEVEL 2 ($n = 100$). "FRAME" = GATE-FRAME CLIPS, "COLL." = TOTAL COLLISIONS.

Configuration	Success	Mean [s]	Frame	Coll.
Full (all on)	88.0%	7.37	4	5
− Gate-track boost	83.0%	7.46	10	8
− Curvature cap	88.0%	7.15	6	7
− Caution mode	79.0%	6.13	15	17
All off (raw MPCC)	62.0%	5.97	25	27

2.5 m/s, no re-tuning) on the real drone with a motion-capture state estimate. Across eight consecutive flights the drone completed the track *every time*, with lap times of $\{8.20, 8.53, 8.54, 8.71, 8.72, 8.79, 8.84, 9.24\}$ s (Table III). The mean of 8.70 s lies within 0.2 s of the Level-3 simulation mean (8.51 s), a small sim-to-real gap: the physical laps are marginally slower and more variable, consistent with unmodelled aerodynamics, battery sag, and estimator latency. The RTI solve held the 50 Hz deadline apart from occasional ≤ 10 ms overruns that did not affect completion, so the controller transfers to hardware without modification, validating the simulation results.

D. Ablation study

To quantify each component we perform a leave-one-out ablation (remove one mechanism, keep the rest) and an all-off reference (raw MPCC), all on Level 2 (Table IV).

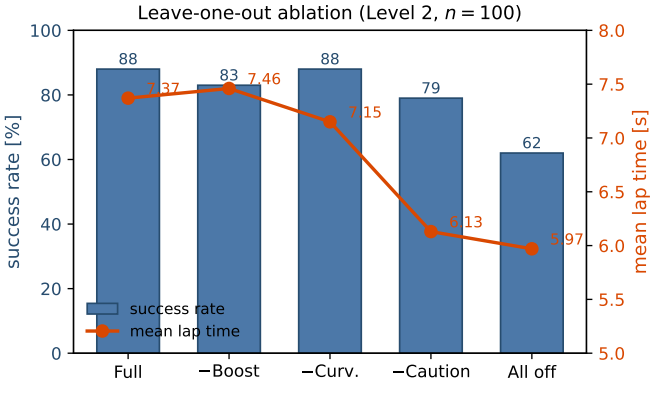

Fig. 5. Leave-one-out ablation on Level 2 ($n = 100$): success rate (bars, left) and mean lap time (line, right). Caution is the dominant robustness lever; removing all rules yields the fast but fragile raw MPCC.

TABLE V
CRUISE-TARGET SWEEP FOR THE FULL CONTROLLER ON BOTH LEVELS ($n = 100$).

	Level 2		Level 3	
V_{tgt} [m/s]	Success	Time [s]	Success	Time [s]
2.0	90.0%	8.08	94.0%	9.17
2.5 (default)	88.0%	7.37	87.0%	8.51
3.0	79.0%	6.83	83.0%	7.95
3.5	74.0%	6.80	72.0%	7.74

The results (Fig. 5) isolate three distinct roles. *Caution* is the dominant robustness lever: removing it drops success by 9 points and nearly triples collisions ($5 \rightarrow 17$), because the drone enters a blind gate at full speed and cannot absorb the replan when the true pose appears at $r_s = 0.7$ m; in exchange it saves 1.24 s, making explicit the speed/robustness trade-off. The *gate-track boost* is a precision lever: removing it roughly doubles gate-frame clips ($4 \rightarrow 10$) and costs 5 points of success at essentially no change in lap time, confirming that tightening q_c near gates buys frame clearance almost for free. The *curvature cap* is neutral in success at our conservative V_{tgt} and is in fact marginally faster when removed; its value is as a *speed-scaling safety margin* that becomes essential once V_{tgt} is raised, keeping a fast configuration from overshooting the reversal. Stacking all three converts the raw MPCC (62%/5.97 s) into the full controller (88%/7.37 s): +26 points of success for +1.4 s, which shows that the heuristics, not the base solver, are the source of robustness.

E. Speed/robustness trade-off

The ablation shows the geometric rules improve the underlying success/lap-time trade-off, rather than merely exchanging one for the other. Could a single scalar, the cruise target V_{tgt}, achieve the same on its own? Table V sweeps it for the full controller. Lowering it to 2.0 m/s trades 0.7 s of lap time for 2 points of success, i.e. it stays on the same Pareto front. Pushing it past the matched value, however, falls off a cliff: at 3.0 m/s success already drops to 79%, and at 3.5 m/s (well above the soft per-axis velocity bound $v_{\max} = 2.0$ m/s) the optimiser flies permanently against that bound, oversteering into the frames, yet buys almost no extra time ($6.83 \rightarrow 6.80$ s) while success collapses to 74% (collisions rise to 13%). The knob therefore only *slides* the controller along a success/time Pareto front and quickly saturates; it cannot move the front itself. The best trade is obtained by matching V_{tgt} to the actuation limit, which is why we keep $V_{\text{tgt}} = 2.5$ m/s. The identical monotone shape on the held-out Level 3 (Table V) shows this corner-limited behaviour is not an artefact of the single development track.

F. Generalisation to unseen tracks

Because Level 2 is a single fixed track on which the controller was tuned, we repeat the ablation on the fully randomised Level 3 as a held-out test (Fig. 6 and Table VI). The verdict is nuanced. The full adaptation stack still clearly

TABLE VI
HELD-OUT ABLATION ON THE RANDOMISED LEVEL 3 ($n = 100$, SAME SEED AS LEVEL 2 SO THE TRACKS ARE PAIRED).

Configuration	Success	Mean [s]	Coll.
Full (all on)	87.0%	8.51	10
− Gate-track boost	91.0%	8.32	5
− Curvature cap	83.0%	7.86	12
− Caution mode	90.0%	7.46	8
All off (raw MPCC)	81.0%	7.30	17

beats the raw MPCC on unseen tracks (87% vs. 81%, collisions $17 \rightarrow 10$), so the layer generalises *as a whole*. Per component, however, only the *curvature cap* transfers cleanly: removing it costs 4 points on Level 3 (it was neutral on Level 2), matching the intuition that random layouts contain sharper corners where the lateral-acceleration limit actually binds. In contrast, the *gate-track boost* and *caution mode* (the two components whose radii were tuned to Level 2's fixed reversal and gate spacing) do not help on Level 3; removing either even nudges success up (+4 and +3 points), i.e. they are mildly specialised to the development geometry. We keep them because they are strongly positive on the realistic fixed-track setting the controller is ultimately graded on and not harmful in aggregate, but this points to making them geometry-adaptive via the learned weight policy that motivated them.

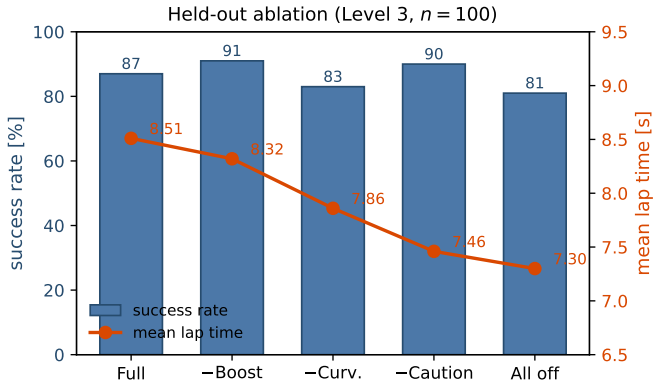

Fig. 6. Held-out ablation on the randomised Level 3 ($n = 100$): success rate (bars, left) and mean lap time (line, right). Unlike Level 2 (Fig. 5), removing the gate-track boost or caution mode does not lower success (these rules are specialised to the fixed track), whereas removing the curvature cap does, and the raw controller is still worst.

G. Failure modes

Table VII breaks the failures down by the nearest physical object at the moment of the crash, aggregated over the full-controller runs of both levels. Gate-frame clips dominate on both tracks and concentrate on gate 2 (the tight 180° reversal) and the final gate, where the true pose is revealed latest and the drone has the least room to correct. Obstacle hits are the second source but stay low on both tracks (4 and 3), so the capsule avoidance keeps clearing the poles even on the randomised Level 3 layouts. Ground and out-of-bounds

TABLE VII
FAILURE BREAKDOWN FOR THE FULL CONTROLLER ($n = 100$ PER LEVEL).

Level	Gate frame	Obstacle	Ground	OOB	Timeout
Level 2	4	4	3	0	4
Level 3	9	3	0	2	1

terminations are rare. The limiting factor is thus *contouring precision at speed in corners*, not straight-line velocity, which each geometric heuristic attacks directly, explaining why they compound so strongly in the ablation of Table IV.

H. Discussion

The task is thus *corner*-limited, not velocity-limited: the scalar cruise target only slides the controller along a Pareto front (Table V), whereas the geometric rules (Table IV) move that front outward, though on held-out random tracks the two geometry-tuned rules lose their edge (Table VI). This motivates the future directions in Section VI.

VI. CONCLUSION

We presented an MPCC-based drone-racing controller that embeds a collision-aware racing line into the prediction model as a function of a progress state, eliminating the reference-runaway failure of time-indexed reference tracking. Three lightweight context-adaptation rules (a curvature speed cap, a near-gate contouring boost, and a caution mode) lift a raw 62% success rate to 88% on a fixed track while retaining 87% on fully randomised tracks, and an ablation study attributes a distinct role to each. A held-out ablation on the randomised tracks further shows that the physics-based curvature rule generalises, whereas the two rules whose radii were tuned to the fixed track are mildly specialised to it, while the stack as a whole still beats the raw controller on unseen geometry. Deployed unchanged on the physical Crazyflie, the controller completed all eight test flights at a 8.70 s mean lap time, within 0.2 s of the simulated Level-3 result, demonstrating that the approach transfers to hardware with only a small sim-to-real gap.

The approach has limitations. Performance is corner-limited: reaching sub-5 s laps would require a structurally better racing line (a smoother reversal with lower curvature admits a higher cornering speed at the same lateral-acceleration budget) rather than higher cruise speed. The adaptation rules, though effective, are hand-designed; the reinforcement-learning weight scheduler that motivated them could not be trained affordably because the embedded `acados` solver is sequential and non-vectorisable, so its online promise remains untested. Future work therefore targets a minimum-curvature racing line and completion of the RL weight policy on parallel (Linux/GPU) infrastructure to make the geometry-specific rules adaptive rather than hand-tuned.

REFERENCES

[1] D. Hanover, A. Loquercio, L. Bauersfeld, A. Romero, R. Penicka, Y. Song, G. Cioffi, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing: A survey," *IEEE Transactions on Robotics*, vol. 40, pp. 3044–3067, 2024.

[2] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Mueller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, pp. 982–987, 08 2023.

[3] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *CoRR*, vol. abs/2108.04537, 2021.

[4] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for near-time-optimal quadrotor flight," *CoRR*, vol. abs/2108.13205, 2021.

[5] D. Lam, C. Manzie, and M. Good, "Model predictive contouring control," in *49th IEEE Conference on Decision and Control (CDC)*, 2010, pp. 6137–6142.

[6] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale rc cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, p. 628–647, 2014. [Online]. Available: http://dx.doi.org/10.1002/oca.2123

[7] M. Krinner, A. Romero, ..., and D. Scaramuzza, "Mpcc++: Model predictive contouring control for time-optimal flight with safety constraints," in *Robotics: Science and Systems (RSS)*, 2024.

[8] R. Penicka and D. Scaramuzza, "Minimum-time quadrotor waypoint flight in cluttered environments," *CoRR*, vol. abs/2202.03947, 2022.

[9] Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing with deep reinforcement learning," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 1205–1212.

[10] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-based model predictive control for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, Oct 2019.

[11] B. Zarrouki, V. Klös, N. Heppner, S. Schwan, R. Ritschel, and R. Voßwinkel, "Weights-varying mpc for autonomous vehicle guidance: a deep reinforcement learning approach," in *2021 European Control Conference (ECC)*, June 2021, pp. 119–125.

[12] M. Zanon and S. Gros, "Safe reinforcement learning using robust mpc," *IEEE Transactions on Automatic Control*, vol. 66, no. 8, pp. 3638–3652, 2020.

[13] M. Mehndiratta, E. Camci, and E. Kayacan, "Automated tuning of nonlinear model predictive controller by reinforcement learning," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct 2018, pp. 3016–3021.

[14] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE International Conference on Robotics and Automation*, May 2011, pp. 2520–2525.

[15] P. Dierckx, *Curve and Surface Fitting with Splines*, 10 2023.

[16] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. v. Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, no. 1, pp. 147–183, 2022.

[17] M. Diehl, H. Bock, and J. Schloder, "Real-time iterations for nonlinear optimal feedback control," in *Proceedings of the 44th IEEE Conference on Decision and Control*, Dec 2005, pp. 5871–5876.

[18] G. Frison and M. Diehl, "Hpipm: a high-performance quadratic programming framework for model predictive control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020.

[19] J. Andersson, J. Gillis, G. Horn, J. Rawlings, and M. Diehl, "Casadi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, 07 2018.

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017.